How I built an AI workflow that waits for *approval*

A pre-renewal resident check-in loop. The architecture, two bugs, and what I'd change before deployment.

n8n / OpenAI / Airtable / Gmail / Telegram

One route to the resident's inbox

OUTBOUND

Outreach → OpenAI draft → Approval → Action router → Gmail

INBOUND

Gmail reply → Match + classify → Manager alert

Airtable · Shared state for both paths

Daily operations + reply polling

Five n8n workflows share Airtable state. Outreach generates drafts; the action workflow owns the only resident send node. Every outbound message needs individual approval.

Give the model a narrow job

Model	Code
Draft wording	Select cohorts
Classify intent	Match identity
Suggest next action	Apply routing rules

Subject token → Exact email → Unmatched alert

The model drafts and classifies. Fixed code owns identity and routing. If a reply cannot be matched, it reaches a person without being assigned to a resident.

Define the output contract

FIELD / RULE	CONSTRAINT
intent	enum
urgency	HIGH | MEDIUM | LOW
confidence	HIGH | MEDIUM | LOW
additionalProperties	false

A closed JSON Schema requires the configured fields and rejects extra properties. Enums constrain intent, urgency and confidence. Workflow code decides the human-review route; schema validity alone doesn't establish correctness.

Make the build reproducible

01 **Shared libraries**
Cohorts, guardrails, routing

02 **Workflow builders + JSON**
Placeholders in Git

03 **Local import**
Resolve local credentials

Shared JavaScript libraries feed builder scripts that generate importable workflow JSON. Exports keep placeholders; local import resolves identifiers and credentials. Git holds the reviewable source and exports.

Validate, then retry once

Generate → Check draft

PASS

Await approval

FAIL

Retry once

↓

Check again

Pass → Await approval

Fail → REVIEW_FAILED

JavaScript checks the draft after generation. Failures return their violations for one retry. A second failure becomes REVIEW_FAILED, visible to the manager without an approve button.

The facts changed after validation

BEFORE

Check → **Edit** → **Send**

Unchecked gap

AFTER

Check → **Edit** → **Recheck** → **Send**

Validate what will actually send

A manager edited an already-checked draft. The fix: validate the current wording again immediately before sending. Keep draft-time checks as well.

Skip was a state-changing action

BEFORE

SENT → Skip → ~~**PENDING**~~

Eligible again

AFTER

SENT → Skip Refused → **SENT**

Skip could reset an already-sent record and make it eligible again. I restricted it to pending review states. A sent record now refuses Skip and keeps its history.

Model failure becomes visible work

OpenAI error

↓

CLASSIFY_FAILED

↓

Manager review

An OpenAI error produces CLASSIFY_FAILED and human review. Low confidence, grievances, sensitive flags and questions also require a person. Suggested messages still need approval; an error cannot become a clean STAYING result.

What the recorded checks established

CHECK	RECORDED RESULT
Draft wording	14 passed
Repeated approval	No second send
Skip after sending	Refused
Reply matching	Matched by token
Classifier generalization	Not established

These synthetic checks verify specific behaviors. They do not establish classifier generalization, production reliability or improved renewals. A held-out reply evaluation is still needed.

Three things I'd change next

01 Evaluation

Held-out labeled replies; measure misses across repeated runs.

02 Send integrity

Atomic claim before sending.

03 Recovery

Reconcile uncertain sends; paginate reads and drain reply backlogs.

Before deployment, I'd prioritize evaluation, send integrity and recovery. The demo's sequential second-tap refusal doesn't prove concurrent approvals or a crash after sending are safe.